
Preprocessing and Visualizing Data

Ordered observations, graphical integrity, and amounts

Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Learning objectives

By the end of this class, you should be able to

- explain how a lag column records an earlier observation without removing dependence;
- calculate a relative change and identify an invalid or ambiguous baseline;
- use a graph and numeric summaries to check different features of the same data;
- diagnose misleading scales, unclear labels, and ineffective visual encodings; and
- build and explain bar plots, dot plots, grouped and stacked bars, and a simple heatmap in R or Python.

Week 2 established a preparation workflow. We checked types and units, preserved original values, and recorded the reasons for transformations. This class adds context to a table and asks how to show what we find. The worked time-series examples make the meaning of a new column explicit. The visualization examples then show why a correct calculation still needs a careful presentation.

Amounts are the main chart family in this class. Class B develops distributions, proportions, and associations and continues the integrated practical. Bring the cleaned data, dictionary, and decision record from Lab 02. Read the current Lab 03 and Assignment 01 instructions in the course repository. Use your chosen language throughout. Keep assessed work private until the deadline and instructor release, then share the permitted portfolio version while retaining the submitted commit and its history.

1 From preparation to a useful view

1.1 What a reduced table leaves out

A simpler table may be easier to inspect, but simplification has a purpose and a cost. Selecting variables removes columns. Combining variables, as in the optional PCA example from Week 2, creates a smaller set of summaries. Sampling selects rows. Binning or merging categories can reduce the number of distinct values without changing the table's row or column count. Keep the original data so that an exploratory choice can be checked or revised.

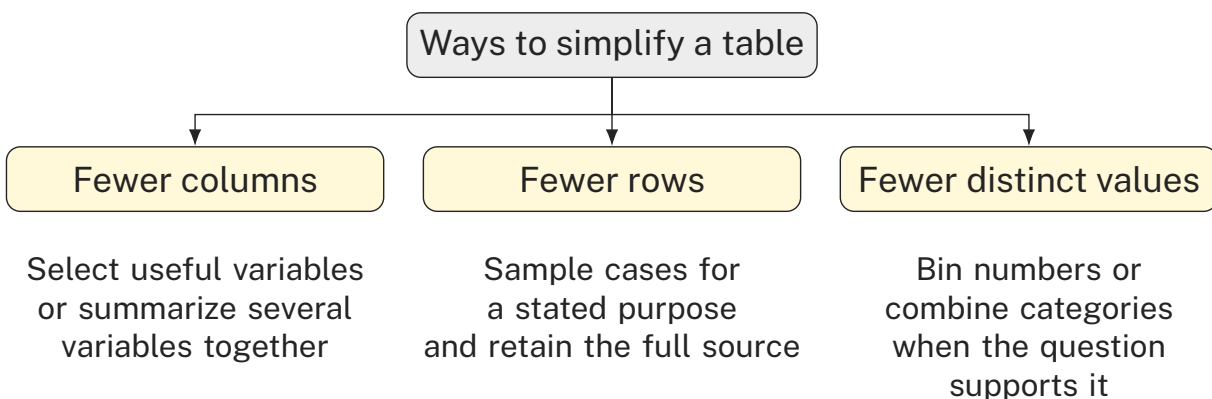


Figure 1. Three ways to simplify a table. Resampling is a broader family of methods; it does not always reduce the number of rows.

Table 1. The dimension or detail changed by each choice.

Change	What it preserves or changes
Fewer columns	Select variables or combine information; detail may be lost.
Fewer rows	Sample for a stated purpose; retain the complete source.
Fewer distinct values	Bin or group values; row and column counts may stay the same.

1.2 Two sampling designs

Simple random sampling gives each possible sample of a fixed size the same chance of selection. Stratified sampling first divides the population into non-overlapping groups, then samples within each group. The diagram uses the same 32 cases throughout, with 12 in group A and 20 in group B. Both designs select eight cases without replacement.

The stratified allocation is proportional here. Three cases come from A and five from B, keeping the population's 12 : 20 ratio. The ordinary sample mean is an unbiased estimate of the population mean under this design, as it is under simple random sampling. If a stratified design deliberately allocates different proportions from the population, combine the stratum means using the population proportions as weights. Stratification can ensure a group appears only when the chosen allocation gives that group at least one case. Rounding a small sampling fraction can still leave a rare group out.

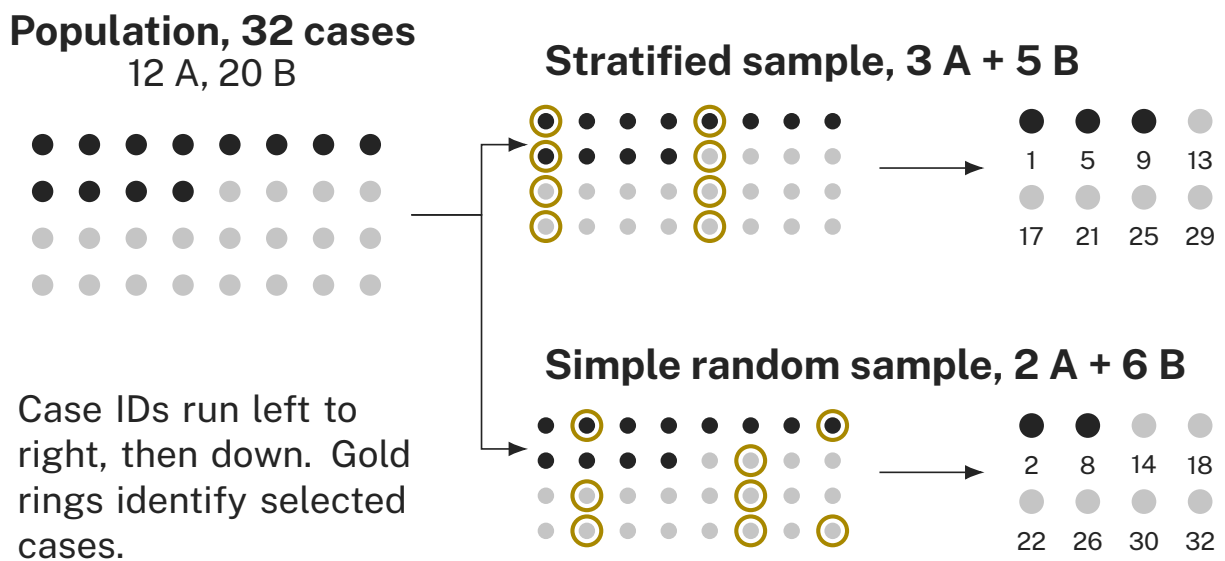


Figure 2. Each gold ring identifies a selected case. The output copies those case IDs unchanged. The two displayed selections illustrate possible samples; they are not claimed to be the output of a particular random seed.

Table 2. The explicit eight-case designs used in this example.

Design	How eight cases are chosen
Simple random	Select eight from all 32, without replacement.
Stratified	Select three from group A and five from group B, without replacement within each group.

For perspective, suppose each independent draw has a 3% chance of belonging to a rare group. The probability that none of 100 draws belongs to that group is $0.97^{100} \approx 0.0476$. This is an independent-draw calculation, also useful as a large-population approximation. Sampling without replacement from a small finite population needs a different calculation. The practical lesson is to check the resulting group counts rather than assuming that a random sample includes every group.

1.3 Run the examples in one language

The following setup is used by every later code section in these notes. Run the sections in order in R or Python. R uses ggplot2 and tidy; Python uses NumPy, pandas, and Matplotlib. Use the installation workflow from Week 1 if a package is missing. Plotting examples write named PDF files into your working directory. All small teaching inputs are defined in the code. Datasaurus is read directly from a pinned public data file and needs an internet connection.

R

```
library(ggplot2)
blue <- "#1F5C8B"
gold <- "#FFD400"
```

Python

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.colors import LinearSegmentedColormap

blue, gold = "#1F5C8B", "#FFD400"
```

Both implementations below define the entire population and the sample allocation. A seed makes a run repeatable within its implementation. The same seed does not guarantee the same selected IDs across R and Python, because the random-number and sampling procedures can differ. Compare the design and group counts, not an assumed cross-language identity of the draw.

R

```
population <- data.frame(id=1:32, group=rep(c("A", "B"), c(12, 20)))
set.seed(5)
simple <- population[sample(nrow(population), 8), ]
stratified <- rbind(
  population[sample(which(population$group == "A"), 3), ],
  population[sample(which(population$group == "B"), 5), ]
)
print(table(stratified$group))
# A: 3, B: 5. Sampling is without replacement.
```

Python

```
population = pd.DataFrame({
    "id": np.arange(1, 33),
    "group": ["A"] * 12 + ["B"] * 20,
})
simple = population.sample(n=8, random_state=5)
stratified = pd.concat([
    population[population["group"] == "A"].sample(n=3, random_state=5),
    population[population["group"] == "B"].sample(n=5, random_state=5),
])
print(stratified["group"].value_counts().to_dict())
# B: 5, A: 3. Sampling is without replacement.
```

1.4 Optional PCA extension

PCA forms new numeric variables by combining existing ones. In a scatterplot, imagine turning the axes so that the first axis follows the direction of greatest variation in the chosen scale. Keeping only that coordinate gives a simpler view while discarding variation in other directions. The meaning of the new axis and the detail lost both matter. High variation alone does not establish scientific relevance.

This is a continuation of the optional Week 2 example, not a prerequisite for Lab 03. The following two resources are also linked in the slides. The first supports the geometric idea; the second adds mathematical background. Covariance matrices and their decomposition are not new practical requirements here.

- Powell and Lehe, PCA explained visually. Explore how the direction of greatest variation changes as the points move.
- Jaadi, A step-by-step PCA explanation. Optional mathematical background after the geometric idea is clear.

1.5 Where this class sits in the course

Feature engineering belongs to preparation, while visualization also supports exploratory data analysis. A plot can expose a preparation issue, and a better-defined variable can make a plot more informative. The course map keeps the same stages and geometry used in the preceding weeks.

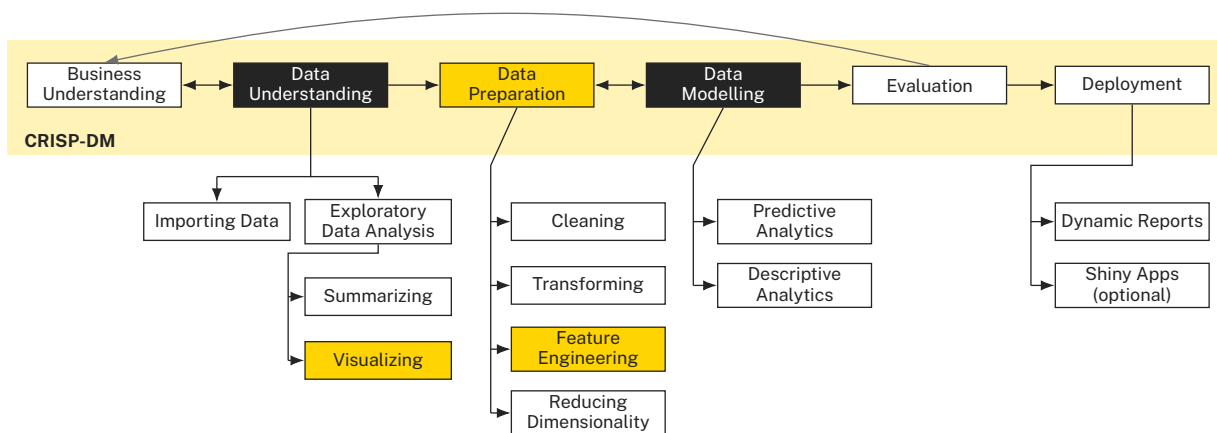


Figure 3. Today's emphasis is feature engineering and visualization.

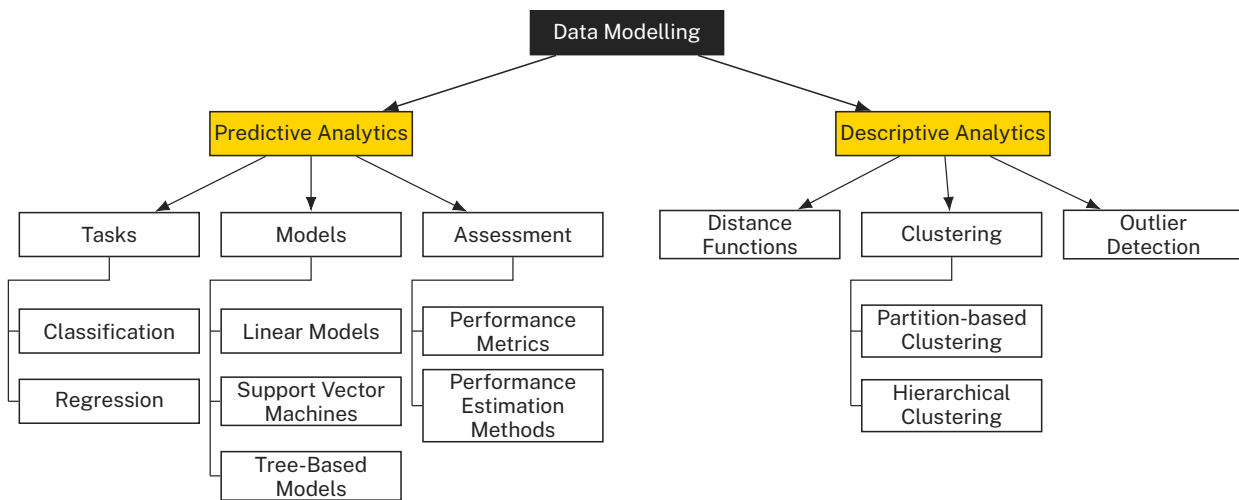


Figure 4. Later course stages provide orientation. Model fitting, model evaluation, and clustering are taught in their scheduled weeks.

2 Record meaningful context in a new column

2.1 What a row does and does not tell us

A row identifies an observation, and its columns record variables. Neither fact guarantees independence. Measurements taken a minute apart at the same sensor may resemble one another. Repeated observations from a person share a subject identity. Nearby places may share environmental conditions. Preserve the relevant time, subject, and location information instead of expecting the current numeric values to identify that context on their own.

A derived variable can express a relationship that matters to the question. An amount per person uses two columns already present. A lag column records an earlier value. A relative change compares successive values on a meaningful baseline. Each derived variable needs a definition, valid inputs, and a clear reason for inclusion in the analysis.

Columns record measured variables
Rows identify observations

Time order and subject identity are part of the data's meaning. Keep both explicit.

Figure 5. A table structure does not establish independence between rows.

There are two complementary responses to ordered or related observations. We can use an analysis designed to account for their order and dependence. We can also represent relevant context explicitly in columns. For example, an earlier reading can be placed next to the current reading for comparison. These choices can be used together. Adding lags does not remove serial dependence, and it does not by itself justify applying a method that assumes independent observations.

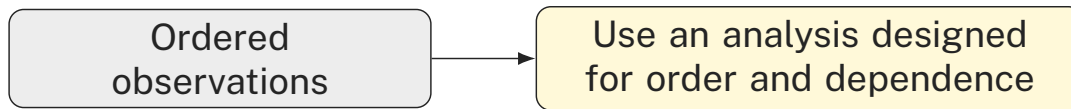
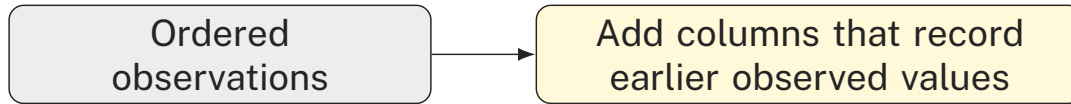


Figure 6. One route is to choose an analysis that represents order and dependence.



The new columns
preserve context.
Dependence can remain.

Figure 7. Another route records observed history explicitly. We practise this representation before studying models.

2.2 Build a time delay embedding

A *lag* is an earlier value in an ordered sequence. Let $Y(t)$ denote the value at time t . The column $Y(t-1)$ records the preceding value, and $Y(t-2)$ records the value two intervals earlier. A table containing the current value and several lags is sometimes called a *time delay embedding*.

Consider one subject observed at seven equally spaced times. The values are 4, 5, 6, 7, 8, 9, 10 in chronological order. With the current value and three lags, the first complete row occurs at time 4. The original seven-row sequence therefore yields four complete rows. In general, a complete sequence of n values with k columns, including the current value, gives $n - k + 1$ rows when $1 \leq k \leq n$.

Table 3. The same chronological lag rows produced by both examples.

Time t	$Y(t)$	$Y(t-1)$	$Y(t-2)$	$Y(t-3)$
4	7	6	5	4
5	8	7	6	5
6	9	8	7	6
7	10	9	8	7

R's `embed` directly constructs the matrix. The numeric literal vector used here stores doubles, so the resulting matrix also stores doubles. In pandas, `shift` leaves missing entries where earlier rows do not exist; `dropna` removes the three incomplete rows in this fully observed example. A real sequence with internal missing values requires its own documented handling rule.

R

```

# One subject, equally spaced times, already in chronological order.
y <- c(4, 5, 6, 7, 8, 9, 10)
lags <- embed(y, dimension=4)
colnames(lags) <- c("Y", "lag1", "lag2", "lag3")
print(lags)
# Rows 7,6,5,4; 8,7,6,5; 9,8,7,6; 10,9,8,7.
  
```

Python

```
# One subject, equally spaced times, already in chronological order.
y = pd.Series([4, 5, 6, 7, 8, 9, 10], dtype=float)
lags = pd.DataFrame({
    "Y": y, "lag1": y.shift(1),
    "lag2": y.shift(2), "lag3": y.shift(3),
}).dropna()
print(lags.to_numpy())
# Rows [7,6,5,4], [8,7,6,5], [9,8,7,6], [10,9,8,7].
```

Before creating lags, sort by time within each subject and decide what one interval means. A row shift is a fixed-duration lag only for equally spaced observations. Never allow the final reading from one subject to become the first lag for another. A geographic identifier is not a time order or a definition of spatial neighbourhood.

Overlapping lag rows still share observations. If these features are later used for prediction, each row may use only information that would have been available at that time. Already observed earlier values can be legitimate history, including values observed before an evaluation period begins. Future values or summaries computed using unavailable future observations are different. Formal training and evaluation procedures are introduced later in the course.

2.3 Compare successive amounts through relative change

For an amount with a meaningful ratio scale and a nonzero previous value, the fractional relative change is

$$r_t = \frac{Y_t - Y_{t-1}}{Y_{t-1}}.$$

A change from 6 to 9 gives $r_t = (9 - 6)/6 = 0.5$, or a 50% increase. The change in the original unit is 3. These answer different questions. One asks how much was added; the other asks how large the increase was relative to the starting amount.

$$6 \longrightarrow 9$$

$$(9 - 6)/6 = 0.5 = 50\%$$

Figure 8. The denominator is the previous amount, not the current amount.

A zero previous value makes this fraction undefined. A missing value makes the pair incomplete. Negative values need a domain-specific interpretation, and a scale without a meaningful zero, such as temperature in degrees Celsius, does not support the usual percentage-growth interpretation. These checks come before the calculation.

The worked sequence is $Y_t = 4(1.5)^t$ for $t = 0, \dots, 5$. Its levels increase, but every successive proportional change equals 0.5. For a different example, first differences of an exactly linear sequence are constant. Neither fact implies that differencing removes arbitrary trends, seasonal patterns, or dependence from real data.

R

```
y <- 4 * 1.5^(0:5)
stopifnot(all(!is.na(y)), all(y > 0))
change <- diff(y) / head(y, -1)
print(change) # Five values equal to 0.5.
print(100 * change) # 50 percent each interval.
```

Python

```
y = pd.Series(4 * 1.5 ** np.arange(6))
assert y.notna().all() and (y > 0).all()
change = y.pct_change(fill_method=None)
print(change.tolist())
# Missing first value, then five values equal to 0.5.
print((100 * change).tolist())
# Missing first value, then 50 percent each interval.
```

R's `diff` computes five differences from six inputs. Dividing by `head(y, -1)` pairs each difference with its previous value. pandas' `pct_change` computes fractional changes despite its name. With `fill_method=None`, it does not fill missing observations before calculating. It retains six positions, with the first change missing because no previous value is available. Multiply by 100 only when reporting a percentage.

3 Check what summaries leave out

3.1 Anscombe's quartet

A mean describes location, and a standard deviation describes spread. Pearson's correlation r , between -1 and 1 , describes the direction and strength of a linear association. None of these summaries alone specifies the shape of a scatterplot. Anscombe's four datasets provide a compact demonstration. Each contains eleven original coordinate pairs.[1]

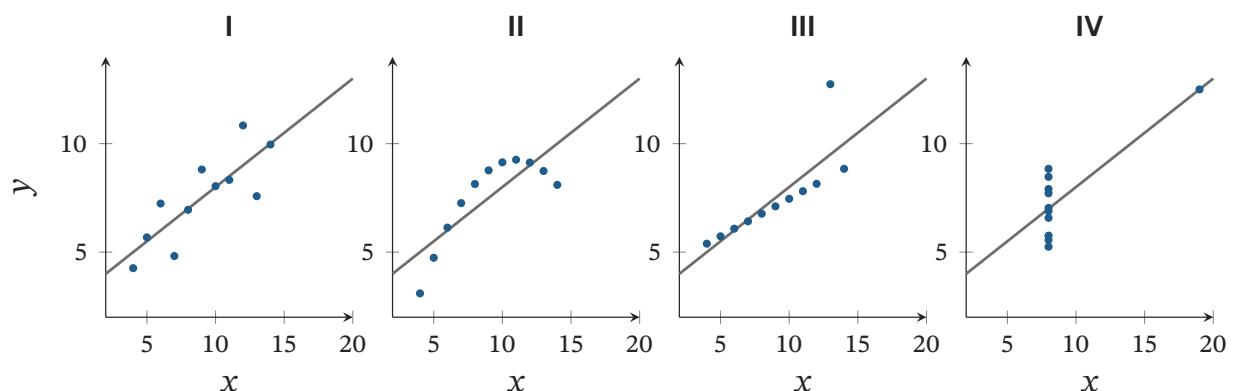


Figure 9. The original quartet on common axes. The grey line is $y = 3 + 0.5x$, a rounded common fitted line shown for comparison. The task is to inspect the points, not to fit a model.

Table 4. The selected summaries agree at this displayed precision, not at every precision.

Summary	Rounded value in each set
Mean x ; sample SD x	9.00; 3.32
Mean y ; sample SD y	7.50; 2.03
Pearson correlation r	0.82
Approximate fitted line	$y = 3 + 0.5x$

The first panel shows points scattered around a roughly straight pattern. The second shows clear curvature. Most points in the third lie close to a straight pattern, with one unusual point above it. In the fourth, ten points share the same x value; the remaining point has much larger x and strongly influences the fitted line. The plots support different descriptions even though the selected summaries are close.

The means and sample standard deviations agree after rounding to two decimals, as does the correlation, 0.82. At three decimals, the fourth correlation rounds to 0.817 while the others round to 0.816. Other summaries, such as medians and ranges, need not agree. The lesson concerns the limits of these selected summaries, not an assertion that every possible calculation gives the same result.

R supplies the original values in `datasets::anscombe`. The Python setup defines all 44 coordinate pairs in a dictionary, one named entry per panel. R uses a list of four tables; `lapply` repeats the same extraction for each pair of columns. The plotting loop then visits each dataset in turn. Both programs draw the same rounded line explicitly; the detailed study of linear fitting comes later.

R

```
# The 44 original coordinates ship with R.
data(anscombe, package="datasets")
quartet <- lapply(1:4, function(i) {
  data.frame(x=anscombe[[i]], y=anscombe[[i+4]])
})
names(quartet) <- c("I", "II", "III", "IV")
```

Python

```
x = [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5]
quartet = {
    "I": (x, [8.04,6.95,7.58,8.81,8.33,9.96,7.24,4.26,10.84,4.82,5.68]),
    "II": (x, [9.14,8.14,8.74,8.77,9.26,8.10,6.13,3.10,9.13,7.26,4.74]),
    "III": (x, [7.46,6.77,12.74,7.11,7.81,8.84,6.08,5.39,8.15,6.42,5.73]),
    "IV": ([8,8,8,8,8,8,8,19,8,8,8],
           [6.58,5.76,7.71,8.84,8.47,7.04,5.25,12.50,5.56,7.91,6.89]),
}
```

R

```
pdf("class-05-anscombe-r.pdf", width=12, height=3)
par(mfrow=c(1, 4), mar=c(4, 4, 2, 1))
for (name in names(quartet)) {
  d <- quartet[[name]]
  print(c(mean(d$x), mean(d$y), sd(d$x), sd(d$y), cor(d$x, d$y)))
  plot(d$x, d$y, xlim=c(2, 20), ylim=c(2, 14),
        xlab="x", ylab="y", main=name, pch=16, col=blue,
        xaxs="i", yaxs="i")
  abline(a=3, b=.5, col="grey")
}
dev.off()
```

Python

```
fig, axes = plt.subplots(1, 4, figsize=(12, 3), sharex=True, sharey=True)
for ax, (name, (x, y)) in zip(axes, quartet.items()):
    values = pd.DataFrame({"x": x, "y": y})
    print(name, values.mean().round(2).tolist(),
          values.std(ddof=1).round(2).tolist(), round(values.corr().iloc[0, 1], 2))
    ax.scatter(x, y, color=blue)
    ax.plot([2, 20], [4, 13], color="grey")
    ax.set(title=name, xlim=(2, 20), ylim=(2, 14), xlabel="x")
axes[0].set_ylabel("y")
fig.tight_layout()
fig.savefig("class-05-anscombe-python.pdf")
plt.close(fig)
```

3.2 The Datasaurus Dozen

Alberto Cairo created the original dinosaur-shaped dataset. Matejka and Fitzmaurice produced other shapes while preserving similar selected statistics, yielding thirteen sets with 142 points each.[2] The figure uses all 1 846 published coordinates, with the same axis limits and aspect ratio in every panel. These shapes are data, not decorative illustrations of a numerical claim.

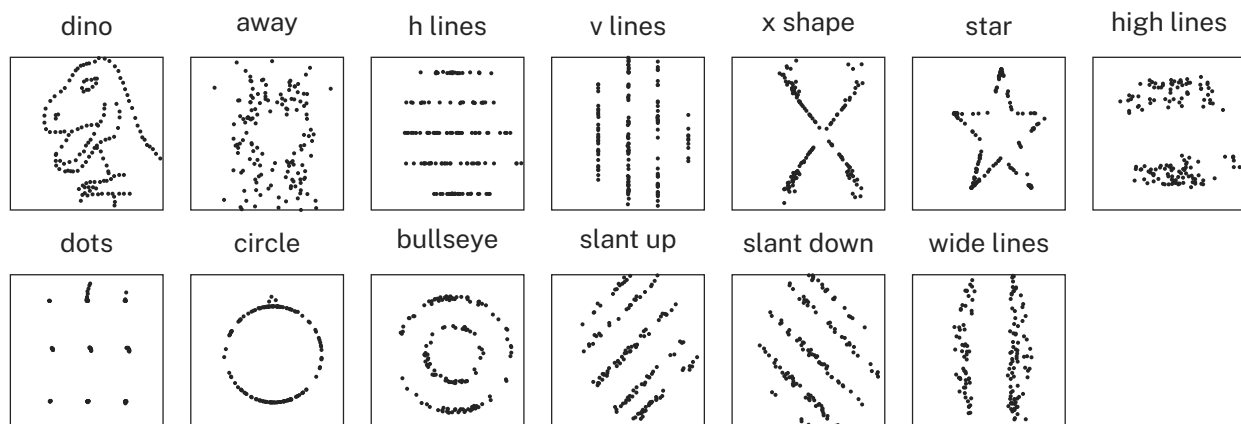


Figure 10. The authentic thirteen datasets. All panels use 0–100 on both axes. Each panel has 142 points.

Table 5. Ranges calculated across the thirteen sets using sample standard deviations.

Summary	Range across 13 sets
Mean x	54.26015 to 54.26993
Mean y	47.83025 to 47.83983
Sample SD x	16.76001 to 16.77000
Sample SD y	26.93000 to 26.94000
Pearson r	−0.06945 to −0.06034

The summaries are nearly equal. They are not all identical under ordinary rounding to two decimal places, so the table reports their ranges instead. The correlations are small and negative, but the circular, crossed, striped, and other shapes remain obvious. A weak linear association does not establish independence or rule out a nonlinear pattern.

The code below reads a pinned revision of the maintainer’s public TSV in either language. Python does not need an R installation or an export from R. The dataset labels, row counts, and coordinate pairs match the file read by these examples.

R

```
revision <- "0496ac15208e9ee4a58ea81a5de46912f095aa15"
url <- paste0("https://raw.githubusercontent.com/jumpingrivers/datasauRus/",
             revision, "/inst/extdata/DatasaurusDozen-Long.tsv")
dino <- read.delim(url)
stopifnot(nrow(dino) == 1846)
print(aggregate(cbind(x, y) ~ dataset, dino, mean))
print(aggregate(cbind(x, y) ~ dataset, dino, sd))
correlations <- by(dino, dino$dataset, function(part) cor(part$x, part$y))
print(correlations)
# Sample SD; values are close, not identical after ordinary rounding.
```

Python

```
revision = "0496ac15208e9ee4a58ea81a5de46912f095aa15"
url = ("https://raw.githubusercontent.com/jumpingrivers/datasauRus/"
      + revision + "/inst/extdata/DatasaurusDozen-Long.tsv")
dino = pd.read_csv(url, sep="\t")
assert len(dino) == 1846
summaries = dino.groupby("dataset")[["x", "y"]].agg(["mean", "std"])
print(summaries.round(3))
for name, part in dino.groupby("dataset"):
    print(name, part["x"].corr(part["y"]))
# Sample SD; values are close, not identical after ordinary rounding.
```

R

```
p <- ggplot(dino, aes(x, y)) + geom_point(size=.6, colour=blue) +
  facet_wrap(~dataset, ncol=5) +
  coord_fixed(xlim=c(0, 100), ylim=c(0, 100), expand=FALSE) +
  theme_minimal()
ggsave("class-05-datasaurus-r.pdf", p, width=12, height=7)
```

Python

```
fig, axes = plt.subplots(3, 5, figsize=(12, 7), sharex=True, sharey=True)
for ax, (name, part) in zip(axes.flat, dino.groupby("dataset", sort=True)):
    ax.scatter(part["x"], part["y"], s=5, color=blue)
    ax.set(title=name, xlim=(0, 100), ylim=(0, 100), aspect="equal")
for ax in axes.flat[13:]:
    ax.set_visible(False)
fig.tight_layout()
fig.savefig("class-05-datasaurus-python.pdf")
plt.close(fig)
```

For each published example, state one feature visible in the plot but absent from the displayed summaries. Then state one quantity that the summaries communicate more precisely than the plot. Both forms of evidence have a role.

4 Choose a chart from a question

The question determines the comparison, while the variable type, units, and meaningful order constrain the encoding. A numeric value can be an amount to compare between groups, an observation in a distribution, a part of a total, or one coordinate in an association. Begin by stating the question in a sentence, then choose the variables and chart that answer it.

Table 6. Five useful question families. This class develops amounts; Class B develops the remaining families.

Question	Starting points
How much?	Bar, dot, grouped or stacked bars; numeric cells in a heatmap.
How are values distributed?	Histogram, density, boxplot, violin, ECDF.
What fraction of a whole?	Proportional bars, pie.
How do two variables relate?	Scatterplot; a line when order has meaning.
How do several variables relate?	Scatterplot matrix, parallel coordinates.

Amounts, this class

Bar and dot plots
Grouped and stacked bars
Heatmaps

Distributions, Class B

Histograms and density plots
Boxplots and violin plots
ECDF plots

Figure 11. Amounts are the main focus here. Distribution displays are developed in Class B.

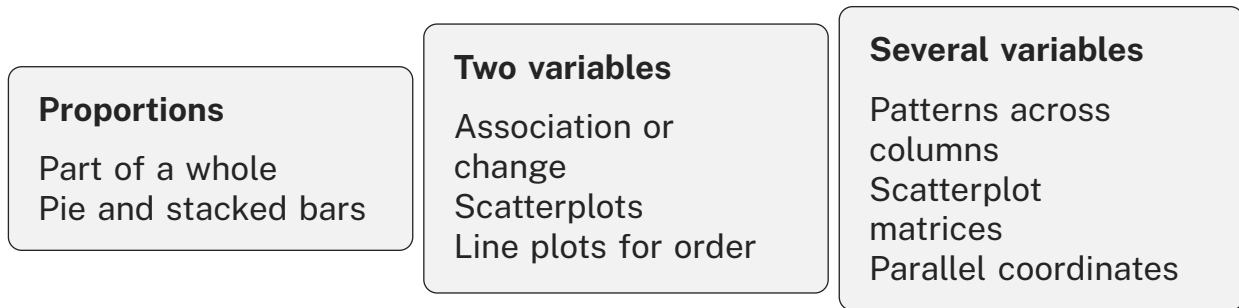


Figure 12. Further chart families, also developed in Class B. A line needs a meaningful order or connection between observations.

4.1 Defaults are choices made by software

A call that appears to request “a histogram” also chooses break locations, treatment of missing values, and axis limits. Under its standard configuration, base R’s `hist` starts from Sturges’ rule and uses convenient break locations; `ggplot2`’s `geom_histogram` starts with 30 bins; `Matplotlib`’s `hist` starts with 10; and `seaborn`’s `histplot` uses `NumPy`’s auto rule. Configuration and library versions can change the resulting choices. Record versions and make consequential choices explicit.

For a controlled comparison, use the same values and the same edges. The following input reuses the eight numbers from Week 2, with one missing entry added. Both examples exclude that entry explicitly and report the exclusion in the title. The edges are 0, 10, 20, 30, 40, 50. Intervals are closed on the left, with the final right endpoint included. None of these observations lies on an interior edge, and both programs return counts 6, 1, 0, 0, 1.

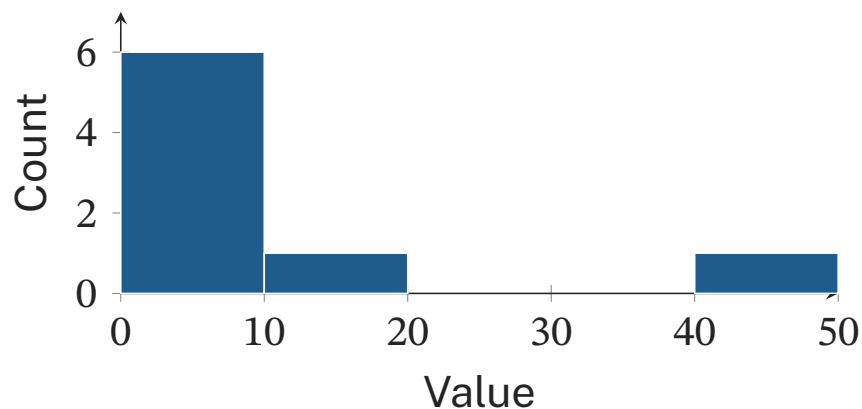


Figure 13. A single fixed dataset and bin specification gives the same counts in both languages.

R

```
values <- c(2, 4, 4, 5, 7, 9, 12, 41, NA)
observed <- values[!is.na(values)]
edges <- c(0, 10, 20, 30, 40, 50)
pdf("class-05-histogram-r.pdf")
h <- hist(observed, breaks=edges, right=FALSE,
          col=blue, border="white", xlab="Value", ylab="Count",
          main="Eight observed values; one missing")
print(h$counts) # 6, 1, 0, 0, 1
dev.off()
```

Python

```
values = pd.Series([2, 4, 4, 5, 7, 9, 12, 41, np.nan])
observed = values.dropna()
edges = [0, 10, 20, 30, 40, 50]
fig, ax = plt.subplots()
counts, _, _ = ax.hist(observed, bins=edges, color=blue, edgecolor="white")
print(counts.astype(int).tolist()) # 6, 1, 0, 0, 1
ax.set(xlabel="Value", ylabel="Count", title="Eight observed values; one missing")
fig.savefig("class-05-histogram-python.pdf")
plt.close(fig)
```

Choosing common bins helps separate a genuine difference in data from a difference in library defaults. The five bins here make that comparison transparent; they are not a claim that one bin choice is ideal for every purpose. Class B returns to distribution displays and the effect of bin width.

5 Make the visual comparison honest

5.1 Bar lengths need their baseline

A teaching redraw compares two rates, 35% and 39.6%. The numeric difference is 4.6 percentage points. Relative to the first value, the increase is

$$\frac{39.6 - 35}{35} \times 100\% \approx 13.1\%.$$

If the bars are clipped at 34%, their visible lengths are 1 and 5.6 percentage points. The second visible bar is therefore 5.6 times as long, even though the second rate is only about 1.13 times the first.



Figure 14. The same two rates. Both show an increase; the clipped baseline exaggerates the apparent magnitude when readers compare bar lengths. This is a teaching redraw, not a reproduced historical broadcast.

Starting bars at zero makes their lengths proportional to the stated amounts. A dot plot can use a narrower, clearly labelled axis because position, rather than length from an implied origin, carries the comparison. That choice still needs context and must not hide relevant values.

5.2 Four practical integrity checks

The following principles adapt Tufte's discussion of graphical integrity into checks for our own figures.[3] They apply to exploratory work as well as presentation. If a plotting choice changes the apparent conclusion, document the choice and inspect an alternative.

Table 7. Questions to ask before accepting a plot.

Match visual magnitude to numeric magnitude.	For bars, begin at zero so length represents the stated amount.
Label the comparison completely.	Give the variable, unit, population, time period, and source.
Show variation in data, not decoration.	Keep scales and encodings consistent across comparable panels.
Give enough context to interpret the result.	Report exclusions and avoid selecting a range that conceals the pattern.

6 Choose visual encodings deliberately

6.1 Magnitude and category identity are different tasks

An encoding maps a value or category to a visible property. Position, length, area, angle, colour intensity, hue, and shape all have uses. For precise comparisons of numeric magnitude, aligned positions and lengths are usually easier to judge than area or angle. The same ranking does not apply to identifying categories. Hue, shape, line style, and text can distinguish groups effectively without implying that one group is larger than another.

Compare numeric values

Aligned position and length usually support more precise comparisons than area, angle, or intensity.

Identify groups

Use hue, shape, line style, or text to distinguish groups. Categories do not need a magnitude ranking.

Figure 15. Choose the encoding for the comparison the reader needs to make.

Table 8. Magnitude comparisons and categorical identification need different cues.

Task	Useful encodings
Precise magnitude	Aligned position or length with a labelled scale.
Approximate magnitude	Area, angle, or intensity when they suit the question.
Category identity	Hue, shape, line style, and direct labels.

The next figure represents the values 2, 4, and 8. Aligned dots and bars make their ratios easy to compare. The circles encode the same numbers through area, so their radii must grow with the square root of the value. Doubling a radius would quadruple area. A pie chart similarly uses angle and area to represent parts of a whole; it may suit a simple part-to-whole question, while aligned bars often make close comparisons easier.

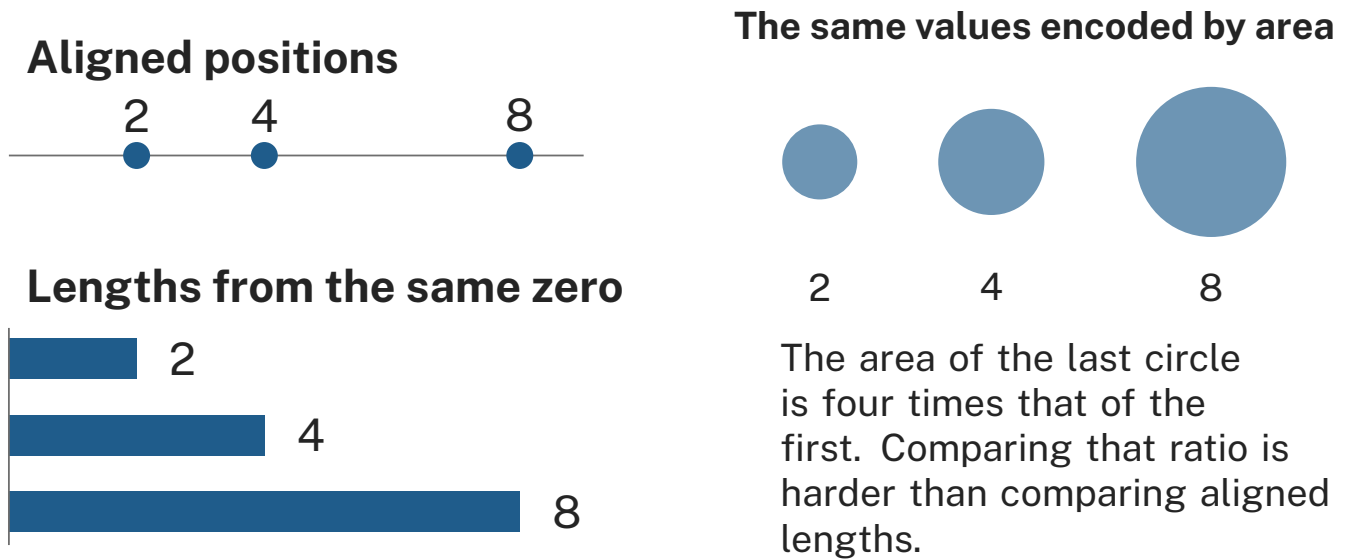


Figure 16. The same three values shown through position, length, and area. Aligned positions make the ratio between the values easier to compare.

6.2 Use more than colour to convey meaning

A chart can retain colour and still give readers another way to identify a group. Direct labels, different shapes, and distinct line styles help when hues are difficult to distinguish or the chart is printed in grayscale. The W3C's guidance on colour asks that colour not be the sole carrier of meaning. A palette alone does not establish accessibility; label size, contrast, descriptions, and the final display conditions also matter.

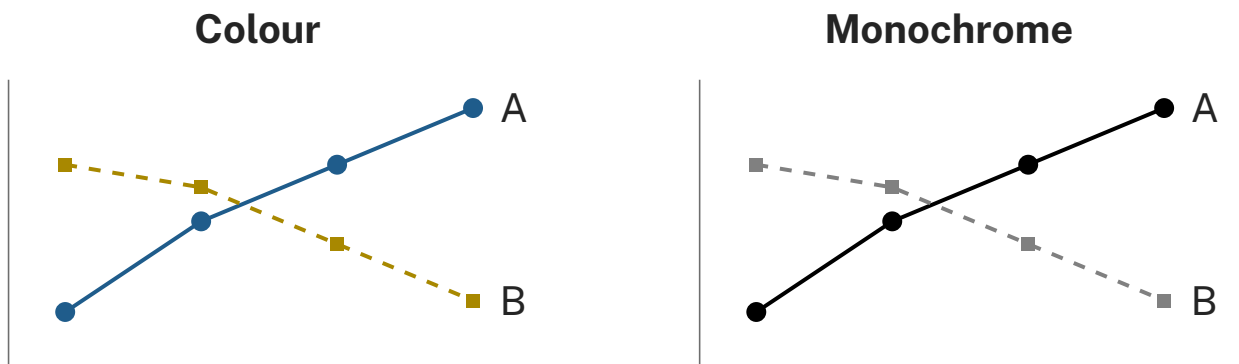


Figure 17. The labelled groups remain distinguishable in colour and monochrome. This comparison is not a physiological simulation of any particular colour-vision condition.

Here group A has circles and a solid line, while group B has squares and a dashed line. The endpoint labels also identify them directly. Inspect the plot at the size at which readers will see it, and provide a caption that states the comparison. A monochrome check is useful evidence about redundant encoding, not proof of accessibility for every reader.

7 Visualize amounts across categories

7.1 Define the data before plotting

The NBA example records regular-season wins for the five Atlantic Division teams over four seasons. These are historical values and do not change with the teaching term. Each team-season pair has one already summarized amount. The question for the first chart is which team had more wins in 2018–19.

Table 9. Atlantic Division regular-season wins. Totals sum the four listed seasons, not a single season.

Team	2015–16	2016–17	2017–18	2018–19	Total
Boston Celtics	48	53	55	49	205
Brooklyn Nets	21	20	28	42	111
New York Knicks	32	31	29	17	109
Philadelphia 76ers	10	28	52	51	141
Toronto Raptors	56	51	59	58	224

The values agree with the NBA's historical standings and season records. The historical standings provide the original context. The following setup defines every value used by the bar, dot, grouped, and stacked examples. Every plot and code example uses these same twenty values.

R

```
nba <- data.frame(
  Team=c("Boston Celtics", "Brooklyn Nets", "New York Knicks",
        "Philadelphia 76ers", "Toronto Raptors"),
  `2015-16`=c(48, 21, 32, 10, 56),
  `2016-17`=c(53, 20, 31, 28, 51),
  `2017-18`=c(55, 28, 29, 52, 59),
  `2018-19`=c(49, 42, 17, 51, 58), check.names=FALSE
)
ordered <- nba[order(nba[["2018-19"]]), ]
ordered$Team <- factor(ordered$Team, levels=ordered$Team)
# Small values at the bottom; largest at the top of a horizontal plot.
```

Python

```
nba = pd.DataFrame({
  "Team": ["Boston Celtics", "Brooklyn Nets", "New York Knicks",
          "Philadelphia 76ers", "Toronto Raptors"],
  "2015-16": [48, 21, 32, 10, 56],
  "2016-17": [53, 20, 31, 28, 51],
  "2017-18": [55, 28, 29, 52, 59],
  "2018-19": [49, 42, 17, 51, 58],
})
ordered = nba.sort_values("2018-19")
# Small values at the bottom; largest at the top of a horizontal plot.
```

7.2 Data, mappings, marks, and labels

Both implementations make the same design decisions. Data supply the values. A mapping assigns team to a categorical position and wins to a numeric position. A mark, such as a bar or a dot, represents each amount. Labels, limits, and layout state the context and support reading.

In R, `ggplot` supplies data and mappings, `geom_col` adds a bar layer for already summarized values, and labels, scales, and a theme complete the figure. In Python, a Matplotlib figure contains axes; `ax.barh` adds bars to the axes and axis methods set labels, limits, and title. Python also has layered interfaces, including seaborn's objects interface. These notes use one straightforward route in each language rather than treating a particular syntax as the visual principle itself.

A function that aggregates observations answers a different question from drawing one already summarized value. For example, disabling seaborn's error bars does not disable its aggregation. Here we draw the five supplied totals directly. If the table contains several rows per team, decide how to summarize them before making the same kind of chart.

7.3 Order and orientation solve different problems

Alphabetical order can help someone locate a named category. Ordering nominal categories by amount helps a ranking question. Neither choice makes long names fit under narrow vertical bars. Rotating the tick labels stops them colliding, but the reader then has to read every name at an angle. The first two examples keep that compromise so the orientation problem stays visible.

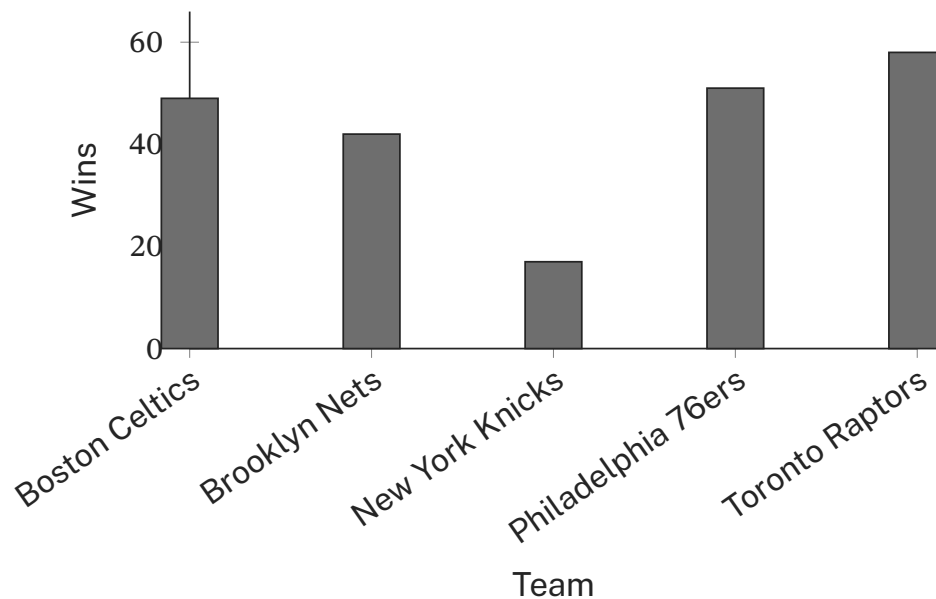


Figure 18. Alphabetical order supports lookup. The long names fit only because they are rotated.

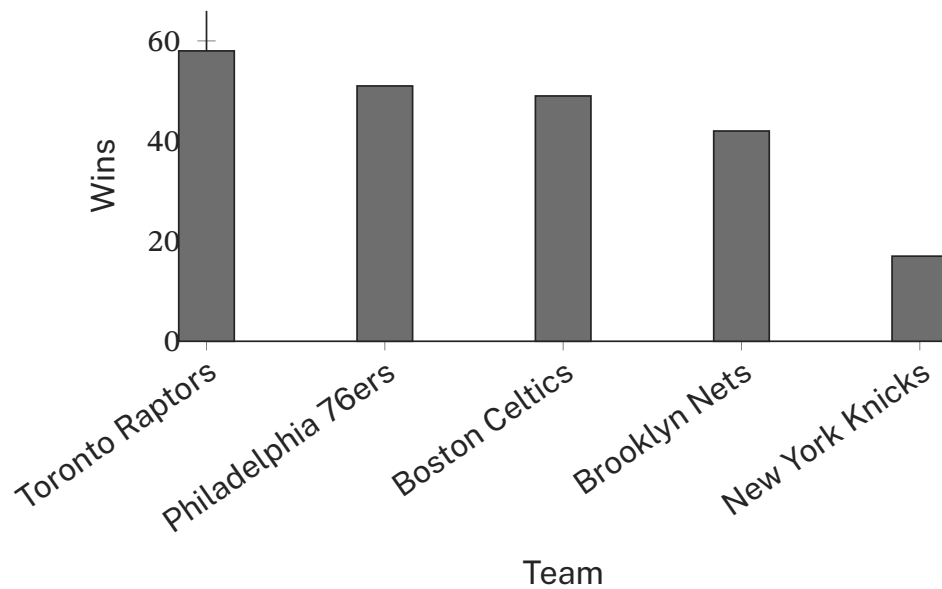


Figure 19. Ordering by wins clarifies the ranking, but the rotated names are still read at an angle. This is an intermediate example to diagnose.

Turning the bars makes room for the names. The improved chart puts the largest amount at the top and retains a zero baseline. Toronto has 58 wins, followed by Philadelphia with 51, Boston with 49, Brooklyn with 42, and New York with 17. If categories have an inherent order, such as an ordinal rating or a sequence of dates, preserve that meaning instead of sorting mechanically by magnitude.

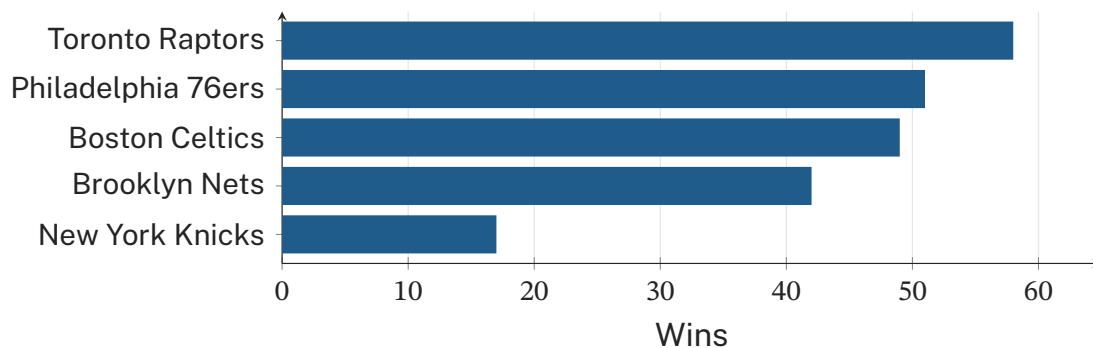


Figure 20. Readable names and a deliberate ranking on the same zero-based scale.

R

```
p <- ggplot(ordered, aes(x=`2018-19`, y=Team)) +
  geom_col(fill=blue) + xlim(0, 65) +
  labs(x="Wins", y=NULL, title="Atlantic Division, 2018-19") +
  theme_minimal()
ggsave("class-05-bars-r.pdf", p, width=7, height=4)
```

Python

```
fig, ax = plt.subplots()
ax.barh(ordered["Team"], ordered["2018-19"], color=blue)
ax.set(xlabel="Wins", title="Atlantic Division, 2018-19", xlim=(0, 65))
fig.tight_layout()
fig.savefig("class-05-bars-python.pdf")
plt.close(fig)
```

The setup orders rows from smallest to largest. R uses those team names as factor levels. Matplotlib's horizontal categorical axis places the first supplied name at the bottom. Thus both examples put Toronto at the top. Explicit ordering avoids an accidental reversal or an alphabetical default.

7.4 A dot can represent the same amount

A dot plot uses position on a common scale. It is often a useful alternative when bar lengths add little or when a carefully labelled narrower axis is needed to inspect close values. The present example retains the bar chart's 0–65 range so that only the mark changes. Each point lies exactly on its team's row; no jitter or aggregation is applied.

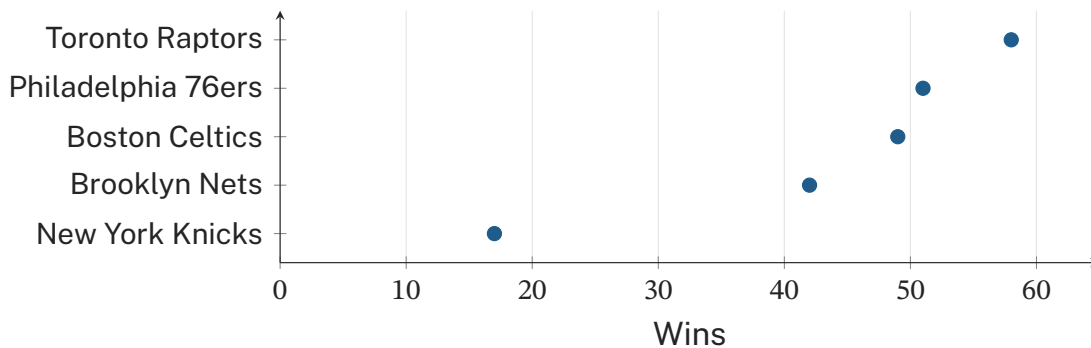


Figure 21. The same five 2018–19 amounts represented by positions.

R

```
p <- ggplot(ordered, aes(x=`2018-19`, y=Team)) +
  geom_point(colour=blue, size=3) + xlim(0, 65) +
  labs(x="Wins", y=NULL, title="Atlantic Division, 2018-19") +
  theme_minimal()
ggsave("class-05-dots-r.pdf", p, width=7, height=4)
```

Python

```
fig, ax = plt.subplots()
ax.scatter(ordered["2018-19"], ordered["Team"], color=blue)
ax.set(xlabel="Wins", title="Atlantic Division, 2018-19", xlim=(0, 65))
fig.tight_layout()
fig.savefig("class-05-dots-python.pdf")
plt.close(fig)
```

7.5 Compare components or compare totals

With four seasons, there are two related questions. Grouped bars make individual seasons easy to compare from a common zero. Stacked bars make the four-season total easy to compare and show each season's contribution. Interior stacked segments do not share a baseline, making close comparisons between those components harder. A stacked bar is appropriate here because wins across distinct seasons can be added meaningfully.

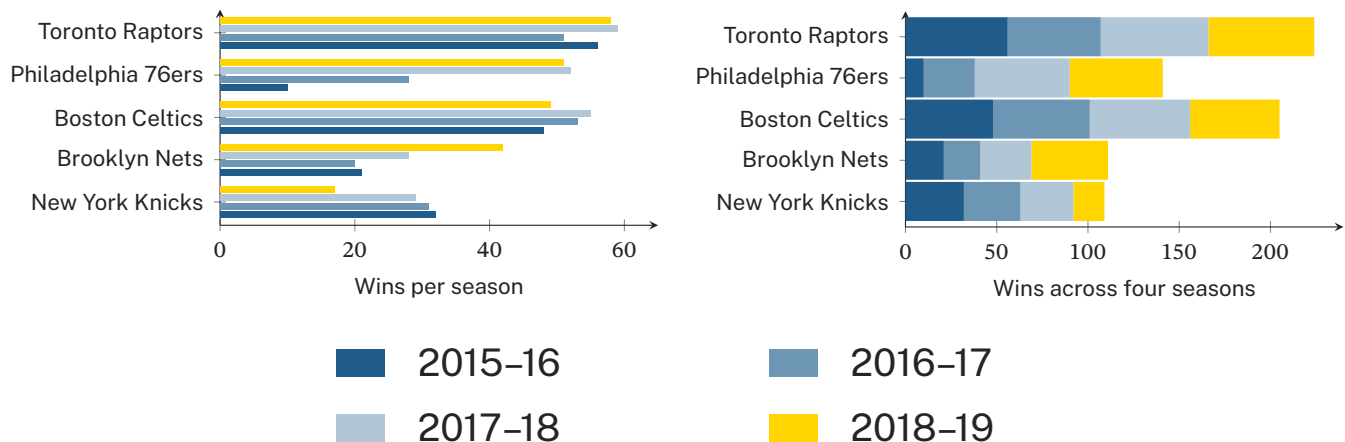


Figure 22. The same twenty values in grouped and stacked form. The category order and season colours are held constant.

Table 10. The comparison changes with the arrangement of the bars.

Display	Comparison supported
Grouped	Compare individual seasons from a common zero.
Stacked	Compare totals and their components; interior segments lack a shared baseline.

The R example reshapes the existing wide table to one row per team and season, following the table-structure ideas from Week 2. Its Season factor fixes the chronological order. The Python example uses the same season order as columns in a wide table. Both shapes represent the same twenty observations. No missing team-season combinations are filled with zero.

R

```
seasons <- names(nba)[-1]
nba_long <- tidyr::pivot_longer(ordered, cols=-Team,
                               names_to="Season", values_to="Wins")
nba_long$Season <- factor(nba_long$Season, levels=seasons)
colors <- c(blue, "#6D95B4", "#B1C6D6", gold)
```

Python

```
seasons = list(nba.columns[1:])
wide = ordered.set_index("Team")[seasons]
colors = [blue, "#6D95B4", "#B1C6D6", gold]
```

First draw grouped bars. The widths and offsets keep the four seasons within each team's row.

R

```
p <- ggplot(nba_long, aes(x=Wins, y=Team, fill=Season)) +
  geom_col(position=position_dodge(width=.8), width=.7) +
  scale_fill_manual(values=colors) + xlim(0, 65) +
  labs(x="Wins per season", y=NULL) + theme_minimal()
ggsave("class-05-grouped-r.pdf", p, width=8, height=5)
```

Python

```
ax = wide.plot.barh(color=colors, width=0.8)
ax.set(xlabel="Wins per season", xlim=(0, 65))
ax.legend(title="Season", ncols=2, loc="upper left", bbox_to_anchor=(1, 1))
ax.figure.tight_layout()
ax.figure.savefig("class-05-grouped-python.pdf")
plt.close(ax.figure)
```

Then stack the same seasons. R's `position_stack(reverse=TRUE)` makes the horizontal segments run left to right in the displayed chronological order. pandas stacks in the supplied column order. The largest four-season total is Toronto's 224 wins; Philadelphia has 141, Boston 205, Brooklyn 111, and New York 109. Those totals answer a different question from the 2018–19 ranking, so the axis label names the four-season period.

R

```
p <- ggplot(nba_long, aes(x=Wins, y=Team, fill=Season)) +
  geom_col(position=position_stack(reverse=TRUE), width=.7) +
  scale_fill_manual(values=colors) + xlim(0, 240) +
  labs(x="Wins across four seasons", y=NULL) + theme_minimal()
ggsave("class-05-stacked-r.pdf", p, width=8, height=5)
```

Python

```
ax = wide.plot.barh(stacked=True, color=colors, width=0.7)
ax.set(xlabel="Wins across four seasons", xlim=(0, 240))
ax.legend(title="Season", ncols=2, loc="upper left", bbox_to_anchor=(1, 1))
ax.figure.tight_layout()
ax.figure.savefig("class-05-stacked-python.pdf")
plt.close(ax.figure)
```

8 Read amounts in a heatmap

8.1 Begin with the cells and their scale

A heatmap maps a matrix of numeric values to coloured cells. Row and column labels identify the pair represented by a cell. The colour key states the numeric mapping. This layout is common in RNA analysis, where rows might identify genes and columns might identify tissues or samples, but the meaning of a colour depends on the measurement and transformation used.

Our example is explicitly synthetic. Its seven imaginary genes and seven tissue labels illustrate the layout; the values are deviations from a chosen reference in arbitrary units. They are not biological

measurements, row-standardized scores, or evidence about real tissues. All 49 classroom values are printed inside the cells and defined in the code.

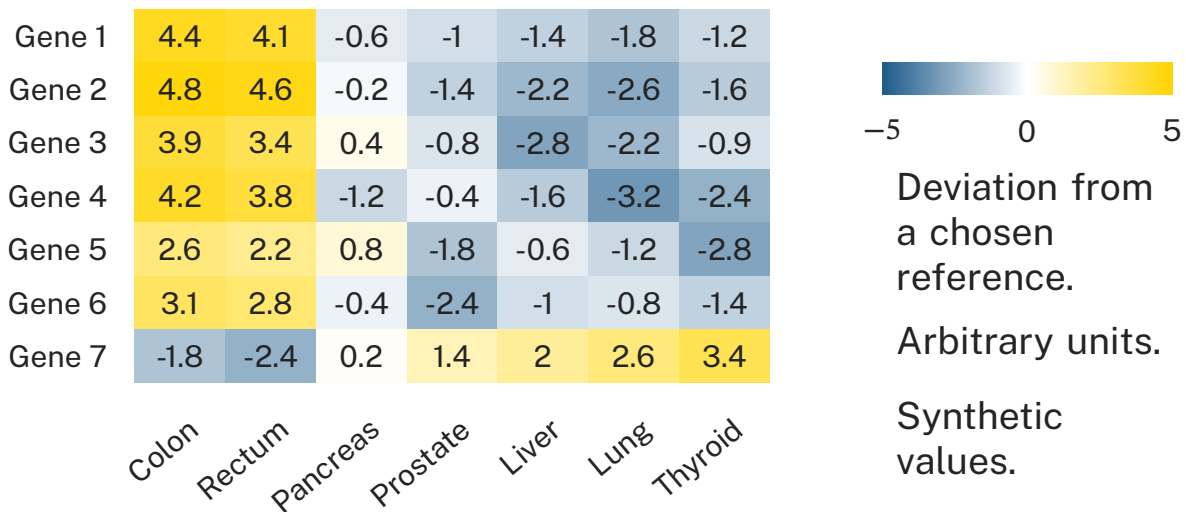


Figure 23. A synthetic matrix with an explicit reference. Blue represents negative deviations, white represents zero, and gold represents positive deviations.

The colour range is fixed at -5 to 5 , with white at zero. White therefore means the chosen reference value. It does not automatically mean the overall data mean, the mean of a row, or a lack of expression. If a later analysis standardizes each row, then zero can represent that row's mean, but that is a different stated transformation. The Week 2 formula for standardization should be applied and checked before making such a claim.

R

```
# Synthetic deviations from a chosen reference, in arbitrary units.
# Rows are imaginary genes; columns are illustrative tissue labels.
values <- rbind(
  c(4.4, 4.1, -.6, -1, -1.4, -1.8, -1.2),
  c(4.8, 4.6, -.2, -1.4, -2.2, -2.6, -1.6),
  c(3.9, 3.4, .4, -.8, -2.8, -2.2, -.9),
  c(4.2, 3.8, -1.2, -.4, -1.6, -3.2, -2.4),
  c(2.6, 2.2, .8, -1.8, -.6, -1.2, -2.8),
  c(3.1, 2.8, -.4, -2.4, -1, -.8, -1.4),
  c(-1.8, -2.4, .2, 1.4, 2, 2.6, 3.4)
)
tissues <- c("Colon", "Rectum", "Pancreas", "Prostate", "Liver", "Lung", "Thyroid")
```

Python

```
# Synthetic deviations from a chosen reference, in arbitrary units.
# Rows are imaginary genes; columns are illustrative tissue labels.
values = np.array([
    [4.4,4.1,-.6,-1,-1.4,-1.8,-1.2],
    [4.8,4.6,-.2,-1.4,-2.2,-2.6,-1.6],
    [3.9,3.4,.4,-.8,-2.8,-2.2,-.9],
    [4.2,3.8,-1.2,-.4,-1.6,-3.2,-2.4],
    [2.6,2.2,.8,-1.8,-.6,-1.2,-2.8],
    [3.1,2.8,-.4,-2.4,-1,-.8,-1.4],
    [-1.8,-2.4,.2,1.4,2,2.6,3.4],
])
tissues = ["Colon", "Rectum", "Pancreas", "Prostate", "Liver", "Lung", "Thyroid"]
```

R

```
cells <- expand.grid(Gene=1:7, Tissue=1:7)
cells$Value <- as.vector(values)
cells$Gene <- factor(cells$Gene, levels=7:1)
cells$Tissue <- factor(cells$Tissue, labels=tissues)
p <- ggplot(cells, aes(Tissue, Gene, fill=Value)) + geom_tile() +
  scale_fill_gradient2(low=blue, mid="white", high=gold,
    midpoint=0, limits=c(-5, 5),
    name="Deviation\n(arbitrary units)") +
  labs(title="Synthetic classroom matrix", x=NULL, y="Gene") +
  theme_minimal() + theme(axis.text.x=element_text(angle=40, hjust=1))
ggsave("class-05-heatmap-r.pdf", p, width=8, height=5)
```

Python

```
palette = LinearSegmentedColormap.from_list("deviation", [blue, "white", gold])
fig, ax = plt.subplots()
image = ax.imshow(values, cmap=palette, vmin=-5, vmax=5)
ax.set_xticks(range(7), tissues, rotation=40, ha="right")
ax.set_yticks(range(7), [f"Gene {i}" for i in range(1, 8)])
fig.colorbar(image, ax=ax, label="Deviation from reference (arbitrary units)")
ax.set_title("Synthetic classroom matrix")
fig.tight_layout()
fig.savefig("class-05-heatmap-python.pdf")
plt.close(fig)
```

8.2 Optional extension, clustered heatmaps

A heatmap does not require clustering. In this example, rows and columns retain their stated order. A clustered heatmap can reorder similar rows or columns and add a branching tree, called a dendrogram, that records the grouping. The tree must be computed from the same data under declared rules; it is not decoration.

A result depends on the chosen measure of similarity, the rule for combining groups, and any scaling applied before comparison. Defaults can differ between libraries. These methods are taught later in the course. For this class, explain the value in one cell, the zero reference, and a visible row or column pattern without making a biological or clustering claim.

9 Bring the decisions into the practical

The Class B meeting develops distributions, proportions, and associations before continuing Lab 03. Bring your prepared dataset and its decision record. Start from a question that your variables can answer, choose an appropriate plot, and document decisions about units, missing values, ordering, scale, grouping, and labels. Follow the current activity for the assessed deliverables.

The lag and growth sequences in this class are worked examples of defining a new variable. They do not require you to invent a time order for an unordered dataset or treat a geographic identifier as a lag. Use derived variables only when their meaning and prerequisites fit your data. Continue in your chosen R or Python workflow.

Can another reader identify the observation, variable, unit, population, and period from the plot and caption? Do the plotted values agree with a numeric check? Is the main comparison still readable without colour? Record any limitation that remains.

Optional further reading

- Wilke, Visualizing amounts. Ordering, orientation, and alternatives to bars.
- Matejka and Fitzmaurice, Same Stats, Different Graphs. Published data behind the thirteen shapes.
- W3C, Use of Color. Pair colour with text, shape, or another cue.
- ggplot2, Bar charts or Matplotlib, Horizontal bars. Read the examples for your chosen language.

References

- [1] F. J. Anscombe. “Graphs in statistical analysis”. In: *The American Statistician* 27.1 (1973), pp. 17–21.
- [2] J. Matejka and G. Fitzmaurice. “Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing”. In: *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*. 2017, pp. 1290–1294.
- [3] E. R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press.